

Multi-tenant Inventory API

Running, testing, and architecture guide

Runtime	Data	Validation
Node.js + Express	Prisma + PostgreSQL	Joi + HTTP tests

REST only - no Docker and no GraphQL

Includes dynamic variants, warehouse stock, atomic transfers, tenant isolation, Postman examples, and TDD verification.

1. Setup without Docker

Requirements: Node.js 22 or newer, npm, and a running PostgreSQL server.

Create two databases

```
createdb -U postgres inventory
createdb -U postgres inventory_test
```

Use your own PostgreSQL administrator role if it is not named postgres. The second database is isolated because tests clear their data between cases.

Configure environment

```
cp .env.example .env

DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/inventory?schema=public"
TEST_DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/inventory_test?schema=public"
PORT=3000
```

Install, migrate, test, and run

```
npm install
npm run db:generate
npm run db:migrate
npm run db:migrate:test
npm test
npm run dev
```

Smoke test

```
curl http://localhost:3000/health
```

2. Architecture and tenant boundary

The tenant is resolved from x-tenant-api-key middleware. Clients never provide the tenantId used by services. Every tenant-owned query includes tenantId, and composite foreign keys prevent cross-tenant relationships.

Domain model

Entity	Ownership and responsibility
Tenant	Owns catalog, warehouses, stock, ledger, and transfers.
Product	Defines a merchant-specific JSON variant schema.
ProductVariant	Sellable SKU whose attributes satisfy its product schema.
Warehouse	Tenant-owned physical inventory location.
StockLevel	One balance per tenant, variant, and warehouse tuple.
InventoryAdjustment	Immutable ledger entry with delta and balanceAfter.
StockTransfer	Atomic movement between two warehouses.

Request flow

Step	Responsibility
Route	Connects the HTTP method and path to a controller.
Middleware	Authenticates the API key and resolves tenantId.
Controller	Validates HTTP input and formats the response.
Service	Applies tenant-scoped business rules and transactions.
Prisma	Executes PostgreSQL queries and migrations.

3. Dynamic product variants

A merchant defines attribute names and rules without changing the relational schema. Joi validates both the schema definition and each variant payload.

Create a product schema

```
POST /products
x-tenant-api-key: TENANT_API_KEY

{
  "name": "Phone",
  "sku": "PHONE",
  "variantSchema": {
    "color": {
      "type": "string",
      "required": true,
      "values": ["black", "white"]
    },
    "storage": {
      "type": "number",
      "required": true,
      "min": 128,
      "max": 512
    },
    "refurbished": { "type": "boolean" }
  }
}
```

Create a matching variant

```
POST /products/PRODUCT_ID/variants
x-tenant-api-key: TENANT_API_KEY

{
  "sku": "PHONE-BLACK-256",
  "attributes": {
    "color": "black",
    "storage": 256,
    "refurbished": false
  }
}
```

A color outside the allowed values, storage below 128, or an unknown attribute returns HTTP 400 with validation details.

4. Warehouse stock and transactions

Stock is stored independently for each tenant, variant, and warehouse. A guarded database update prevents negative quantities and integer overflow, including concurrent withdrawal attempts.

Add opening stock

```
POST /inventory/adjustments
x-tenant-api-key: TENANT_API_KEY
```

```
{
  "variantId": "VARIANT_ID",
  "warehouseId": "JAKARTA_ID",
  "delta": 10,
  "reason": "opening stock"
}
```

Transfer between warehouses

```
POST /inventory/transfers
x-tenant-api-key: TENANT_API_KEY
```

```
{
  "variantId": "VARIANT_ID",
  "sourceWarehouseId": "JAKARTA_ID",
  "targetWarehouseId": "MEDAN_ID",
  "quantity": 4,
  "reason": "rebalancing"
}
```

The transfer debits the source, credits the target, stores the transfer, and writes two ledger entries in one transaction. Any failure rolls back every change.

Rejected withdrawal example

Operation	Expected result
Current quantity = 5	Stock is available for reads.
POST delta = -6	409 Insufficient stock
GET stock afterward	200 OK, quantity remains 5

5. Postman click-through

Import `output/postman/multi-tenant-inventory.postman_collection.json` and run the collection in order. Do not copy IDs manually.

Request	Saved automatically	Used next
Create tenant	tenantApiKey	Protected endpoints
Create product	productId	Create variant
Create variant	variantId	Stock and summary
Create Jakarta	warehouseId	Stock and transfer
Create Bandung	warehouseId2	Transfer target

Important

- Use Send or Run collection. Opening a saved Example does not execute scripts.
- The collection stores runtime IDs in collection variables.
- If an active environment already defines the same ID, it is refreshed too.
- A unique runId prevents duplicate SKU and warehouse-code conflicts.

Standard response envelopes

```
// Success
{
  "status": "ok",
  "message": "Product created successfully",
  "data": { "id": "..." }
}

// Error
{
  "status": "error",
  "message": "Request validation failed",
  "errors": ["sku is required"]
}
```

6. Verification and submission

Run the following commands before demonstrating or submitting the project.

```
npm test
npx prisma validate
npx prisma migrate status
npm audit --omit=dev
```

Coverage areas

- API-key authentication and tenant isolation
- Dynamic schema definition and variant attribute validation
- Duplicate tenant-scoped SKU and warehouse-code handling
- Multi-warehouse stock and total inventory summary
- Negative-stock rejection and concurrent withdrawal safety
- Atomic transfer success and rollback behavior
- Consistent success and error response envelopes
- Postman saved examples and response-status contract

Architecture decisions

Prisma keeps persistence readable, while PostgreSQL provides relational constraints and transactions. JSON variant schemas provide merchant flexibility without creating per-tenant tables. Composite tenant keys protect ownership at both the service and database layers.

AI-assisted development disclosure

AI tools were used for boilerplate, query review, documentation, and test suggestions. Generated work was reviewed, simplified into route-controller-validation-service layers, and verified through HTTP integration tests. See docs/architecture-report.md for the brief report required by the assignment.

Submission files: README.md, docs/architecture-report.md, this PDF, the Postman collection, Prisma schema and migrations, source code, and tests.